Resumo de Estudo

Assistente de Receitas - Projeto Final IA 2 (Gemini + Function Calling + APIs externas + Front-end)
Integrantes: Matheus Figueira Merlos e João Pedro Marucci Pagluiso

1. O projeto em uma frase

> **Frase para decorar:** O usuário digita ingredientes; o front envia ao backend Flask (MVC); o Gemini, com function calling, pede funções que consultam a TheMealDB (receitas) e a TheCocktailDB (bebida, se pedida); o resultado volta em JSON e o front mostra uma receita real, traduzida, com foto.

Fluxo em 8 passos

1. Usuário escolhe ingredientes, tipo de refeição e bebida e clica em **Criar Receita**.
2. O front (`script.js`) faz `POST /api/receitas/gerar` com JSON `{ingredientes, tipoRefeicao, bebida, evitar}` .
3. O **Controller** valida o corpo (400 se inválido) e chama o **Model**.
4. O Model envia o pedido ao **Gemini** com a system instruction e as 3 funções (`tools`).
5. O Gemini **pede** `buscar_receitas_por_ingredientes` ; o Python executa, consulta a TheMealDB e devolve o resultado ao Gemini.
6. O Gemini escolhe uma receita e pede `ver_receita` (e `sugerir_bebida` , se pedida).
7. O Gemini responde com um **JSON**; o Model valida, normaliza e acrescenta **foto e id reais** da API.
8. A **View** devolve `{sucesso, dados, funcoes}` e o front desenha o cartão.

2. Arquivos e o que cada um faz

Arquivo	Função
`backend/app.py`	Cria o Flask, liga CORS, registra o Blueprint, handlers 404/405 em JSON; roda em **127.0.0.1:5000**
`controllers/receita_controller.py`	Rotas `GET /api/status` e `POST /api/receitas/gerar` ; valida entrada; captura erros
`views/resposta_view.py`	Padroniza as respostas JSON (`sucesso` , `erro` , `status`)
`models/receita_model.py`	**Integração com o Gemini**: instrução, loop de function calling, validação, erros, montagem da receita
`models/ferramentas_model.py`	**As 3 funções** que o Gemini chama (APIs externas) e o dicionário `FUNCOES`
`frontend/index.html`	Estrutura: formulário, abas, cartão da receita, rodapé
`frontend/style.css`	Visual laranja (variáveis, flexbox, `.oculto` , `@media` para celular)
`frontend/script.js`	Estado, DOM, `fetch` , abas, receitas salvas (`localStorage`)
`.env` / `.env.example`	Chave `GEMINI_API_KEY` (o real não vai ao Git)

MVC: Model = regras e dados; View = formato da resposta; Controller = recebe, valida e coordena. O front é a interface separada.

3. Function calling (o ponto principal do trabalho)

- **O que é:** o modelo **não executa** nada; ele **pede** que a aplicação execute uma função com certos argumentos. A aplicação executa, devolve o resultado e o modelo continua usando dados reais.
- **As 3 funções:** `buscar_receitas_por_ingredientes(ingredientes: list[str], tipo_refeicao: str)`, `ver_receita(id_receita: str)`, `sugerir_bebida()`.
- **Como o Gemini as conhece:** `tools=list(FUNCOES.values())` em `GenerateContentConfig`. O SDK lê **nome, docstring e type hints** e gera a declaração. A docstring é o que o modelo lê.
- **Loop manual** em `gerar_receita` (até **6 voltas**): chama o Gemini; se há `function_calls`, guarda a resposta do modelo, executa as funções e devolve os resultados; sem pedidos, é a resposta final.
- **Por que manual:** `automatic_function_calling` está **desligado** para registrar as funções chamadas, tratar erros e filtrar receitas já vistas.
- **Guardar a resposta do modelo** (`conteudos.append(resposta.candidates[0].content)`): o Gemini precisa ver que ele pediu a função (nos modelos 3 há assinatura de pensamento).
- **Resultado devolvido:** `types.Part.from_function_response(name=..., response=dicionario)` dentro de uma mensagem `role="user"`. Por isso as funções devolvem **dicionários**.
- **Executar:** `FUNCOES[chamada.name](**chamada.args)` (nome em texto → função; `**` desempacota os argumentos). Erros viram `{"erro": ...}` e o servidor não cai.
- **Prova ao vivo:** o terminal mostra `[Function Calling] Gemini chamou: ...` e o rodapé do cartão lista as funções chamadas.

```
config = types.GenerateContentConfig(
    system_instruction=INSTRUCAO,
    tools=list(FUNCOES.values()),
    automatic_function_calling=types.AutomaticFunctionCallingConfig(disable=True),
)
for _ in range(6):                       # loop do function calling
    resposta = chamar_gemini(conteudos)
    if resposta.function_calls:          # o Gemini PEDIU funcoes
        conteudos.append(resposta.candidates[0].content)
        conteudos.append(executar_funcoes(...))   # executa e devolve os resultados
        continue
    receita = extrair_json(resposta.text)         # resposta final em JSON
    break
```

4. Gemini: instrução, modelos e limites

- **System instruction:** (1) valida se os itens são alimentos, senão devolve `ingredientesValidos: false`; (2) traduz para inglês, singular, e chama a busca **uma vez com a lista completa**; (3) escolhe **uma** receita que não esteja em `evitar` e chama `ver_receita` (nunca inventa ids); (4) bebida só se pedida; (5) sem resultado → `semReceita`; (6) resposta **só em JSON** com título, descrição, tempo, ingredientes, passos, dica, bebida.
- **Anti-alucinação:** o backend só aceita a receita se `ver_receita` foi chamada e encontrou algo (senão **502**); foto e id vêm da API, não do modelo.
- **Modelos:** principal `gemini-3.1-flash-lite` (rápido) e reserva `gemini-3.8-flash`. Se o principal falhar, tenta o reserva; se os dois falharem, traduz o erro.
- **JSON:** `extrair_json` tira cercas de markdown e acha o trecho entre chaves; se falhar, **uma** nova tentativa pedindo só JSON.
- **Tempo/estimativas:** tempo, rendimento e dificuldade são estimativas do Gemini (a API não os fornece).

Limite	Valor
Cada consulta a API externa	10 s
Cada chamada ao Gemini	30 s
Geração inteira (backend)	100 s (erro 504)
Pedido do front (AbortController)	150 s

Limite	Valor
Voltas do loop	6
Itens por pedido / tamanho do item	20 / 40 caracteres

5. A busca de receitas (e o bug do arroz doce)

Bug original: arroz + frango devolvia arroz doce, porque a busca usava só o primeiro ingrediente (arroz) e o modelo pegava uma sobremesa. **Correção em camadas:**

1. A função recebe a **lista inteira**. A TheMealDB gratuita só filtra **um** ingrediente por vez; então consulta cada um e faz a **interseção** (conta em quantas listas cada receita aparece).
2. **Tipo de refeição:** consulta a categoria `Dessert`; se o tipo é Sobremesa mantém só sobremesas, nos outros tipos **remove** sobremesas. Café da manhã prioriza a categoria `Breakfast`.
3. **Honestidade:** devolve `usam_todos_os_ingredientes` e `ingredientes_sem_resultado`; a descrição cita o que ficou de fora.
4. **Parciais:** se nenhuma receita usa tudo, ordena por **raridade** (soma de 1/nº de receitas do ingrediente) e mantém só receitas do **ingrediente principal** (o mais raro).
5. **Resolver nomes:** singular/plural (`tomato` / `tomatoes`) e nomes compostos (`chocolate` → `dark_chocolate`...) usando a lista oficial (cache).
6. **"Outra receita":** o front manda os ids já vistos em `evitar`; o **servidor** os remove dos resultados (não depende do modelo). Acabando, avisa que a lista recomeça.

Exemplo real: `rice` → 30 receitas; `chicken` → 20; interseção inclui *Chicken & chorizo rice pot* (id 53161). O bug antigo trazia *Arroz con Leche* (53522).

APIs: TheMealDB (`filter.php?i=` / `?c=`, `lookup.php?i=ID`, `list.php?i=list`, chave pública `1`) e TheCocktailDB (`random.php`). Dados em **inglês**; o Gemini traduz. Os ingredientes vêm em 20 campos `strIngredient1..20` e `strMeasure1..20`.

6. Erros, validação e segurança

Situação	Código	Mensagem
Corpo inválido, ingredientes ausentes, >20 itens, item >40 caracteres ou com caracteres estranhos	400	Mensagem específica (sem gastar o Gemini)
Itens que não são comida (ex.: cadeira)	422	Estes itens não parecem alimentos
Só temperos ou nada do tipo pedido	404	Mensagem amigável do Gemini
Receita sem ter chamado `ver_receita`	502	Falha ao obter a receita real
Cota do Gemini	429	Aguarde cerca de 1 minuto
Gemini sobrecarregado/timeout	503	Gemini sobrecarregado agora
Chave inválida ou ausente	500	Verifique o .env
Mais de 100 s	504	A busca demorou demais
Rota inexistente / método errado	404 / 405	JSON padronizado
Erro imprevisto	500	Erro genérico (detalhe só no terminal)

- **Chave:** só no `.env` (no `.gitignore`), usada só no backend; `/api/status` informa verdadeiro/falso, nunca a chave.
- **127.0.0.1 (não 0.0.0.0):** com `debug=True` o depurador executa código; não pode ficar exposto à rede.

- **XSS:** o front usa `textContent` (nunca `innerHTML`); testado com `<img onerror>`.
- **Injeção de prompt:** validação de formato, tipo de refeição por lista de permissão, Gemini rejeita o que não é alimento, receita só vale se vier das funções.
- **Limites honestos:** sem login, sem banco (salvas ficam no `localStorage`), sem testes automatizados, base de receitas pequena e em inglês, plano gratuito do Gemini, `resposta_view.inicio` não usada.

7. Front-end em resumo

- **HTML/CSS/JS puros** (sem React). O JS guarda o estado em variáveis e chama `renderizar...()` para redesenhar.
- `el(tag, classe, texto)` cria elementos com `textContent`. `fetch` com `async/await`; `AbortController` cancela após 150 s.
- **Abas** por `data-aba` + classe `.oculto`. **Status** do servidor: verde (online), âmbar (sem chave), vermelho (offline).
- **Submit:** valida, bloqueia o botão, `POST`, checa `resposta.ok` e `json.sucesso`, desenha; `finally` sempre libera o botão. Erros: `AbortError` = demorou; `TypeError` = backend fora.
- **Salvas:** `localStorage` chave `receitas_salvas` (JSON via `JSON.stringify/parse`), com `try/catch`.
- **CSS:** variáveis (`--laranja`...), flexbox, `.oculto {display:none !important}`, `::before` para o marcador laranja, `@media (max-width:480px)` para o celular.

8. Perguntas rápidas (decore as respostas)

1. O que o Gemini faz e o que o Python faz?

O Gemini valida, traduz, decide quais funções chamar, escolhe a receita e organiza o JSON em português. O Python executa as funções, consulta as APIs e valida o resultado.

2. Como provar que é function calling?

Terminal com `[Function Calling] Gemini chamou: ...` e o rodapé do cartão com as funções chamadas.

3. Por que o loop tem limite de 6 voltas?

Para não entrar em loop infinito (gastaria cota e o usuário esperaria para sempre).

4. Por que dois modelos?

O rápido responde melhor; o outro é reserva se houver sobrecarga ou cota. Se os dois falharem, o erro é traduzido para o usuário.

5. Por que uma consulta por ingrediente?

A TheMealDB gratuita só filtra um ingrediente por vez; cruzamos os resultados no código.

6. Como evitam receita inventada?

A instrução manda usar as funções e o backend só aceita a receita se `ver_receita` foi chamada; foto e id vêm da API.

7. Por que POST e não GET?

Os dados vão em JSON no corpo; o POST indica que gera algo novo e não expõe dados na URL.

8. Para que serve CORS?

O navegador bloqueia chamadas entre origens diferentes; o `flask-cors` permite o front falar com o backend.

9. O que é um Blueprint?

Um grupo de rotas em outro arquivo, registrado no `app.py`; mantém o `app.py` pequeno.

10. Por que textContent e não innerHTML?

Trata tudo como texto puro e impede XSS.

11. Onde ficam as receitas salvas?

No `localStorage` do navegador; não passam pelo servidor.

12. O que acontece quando o plano gratuito acaba?

O Gemini devolve 429 e o backend mostra "Aguarde cerca de 1 minuto".

13. Como adicionar uma nova ferramenta?

Criar a função em `ferramentas_model.py` com type hints e docstring e incluí-la em `FUNCOES`.

14. O que significam 400, 404, 422, 429, 500, 502, 503, 504?

400 pedido inválido; 404 não encontrado; 422 dados inválidos (não são alimentos); 429 limite de uso; 500 erro do servidor; 502 resposta ruim de serviço externo; 503 indisponível; 504 tempo esgotado.

15. Quem fez cada parte?

Os dois devem saber responder sobre o projeto inteiro (front, backend, function calling, APIs); a nota é coletiva.

9. Como rodar e checklist

```
cd backend
pip install -r requirements.txt
python app.py            # deixe este terminal aberto
# depois abra frontend/index.html (topo deve mostrar "Servidor online" em verde)
```

- ☐ Arquivo `backend/.env` com `GEMINI_API_KEY=...` (nunca mostrar a chave).
- ☐ Rodar uma geração antes da apresentação (cota do Gemini).
- ☐ Demonstração: **arroz, frango** → receita real, foto e rodapé com as funções; terminal com `[Function Calling]`; "Outra receita"; bebida; "cadeira" (validação); salvar.
- ☐ Abrir `receita_model.py` (loop) e `ferramentas_model.py` (as 3 funções) para mostrar.
- ☐ Não salvar arquivos .py durante a apresentação (o servidor reinicia com `debug=True`).
- ☐ Problemas: servidor offline → rodar `python app.py`; limite de uso → esperar 1 min; porta ocupada → fechar o outro terminal.